

# **DEPARTMENT OF MECHANICAL ENGINEERING**

## **MECHATRONICS & IOT 24MEC51**

### **EXERCISE NO: 4**

|                         |                   |
|-------------------------|-------------------|
| <b>GURUTHARSAN S</b>    | <b>- 24MER011</b> |
| <b>HARISH KUMAR K K</b> | <b>- 24MER015</b> |
| <b>MADHANRAJ K</b>      | <b>- 24MER028</b> |
| <b>SADAAGOPAAN T R</b>  | <b>- 24MER052</b> |

# MECHATRONICS & IoT

## ASSIGNMENT REPORT – 4

### **TITLE:**

Design and develop an IoT-Based HVAC Temperature and Humidity Monitoring System. The system continuously monitors environmental conditions to improve HVAC performance and energy management

### **Done By:**

|                  |            |
|------------------|------------|
| GURTHARSAN S     | - 24MER011 |
| HARISH KUMAR K K | – 24MER015 |
| MADHANRAJ K      | – 24MER028 |
| SADAAGOPAAN T R  | – 24MER052 |

## **Project Overview:**

The IoT-Based HVAC Temperature and Humidity Monitoring System is designed to continuously monitor indoor environmental conditions and help improve HVAC (Heating, Ventilation, and Air Conditioning) performance and energy management.

The system uses a DHT11/DHT22 temperature and humidity sensor to measure the surrounding temperature and relative humidity. An ESP8266 NodeMCU acts as the main controller and provides Wi-Fi connectivity. The measured data can be displayed on a serial monitor and transmitted to an IoT platform for remote monitoring.

The system can identify whether the environment is comfortable, warm, or highly humid and provide an indication when HVAC operation may need adjustment.

The proposed system is useful for:

- Smart buildings
- Offices
- Laboratories
- Classrooms
- Industrial buildings
- Server rooms
- Energy-management applications

## **Problem Statement**

HVAC systems consume a significant amount of energy in buildings. Continuous operation without considering the actual environmental conditions can result in unnecessary energy consumption.

Traditional HVAC systems may not continuously monitor environmental conditions at multiple locations. This can lead to:

- Unnecessary cooling or heating.
- Excessive energy consumption.
- Poor indoor comfort.

- Difficulty in identifying abnormal temperature conditions.
- Lack of remote monitoring.
- Inefficient HVAC operation.

Therefore, an IoT-based monitoring system is required to continuously measure temperature and humidity and provide real-time environmental information for better HVAC performance and energy management.

## Proposed Solution

The proposed system uses a DHT11/DHT22 sensor connected to an ESP8266 NodeMCU.

The temperature and humidity sensor measures the environmental conditions and sends the data to the ESP8266.

The ESP8266 processes the sensor readings and connects to a Wi-Fi network. The measured data can then be transmitted to an IoT dashboard such as ThingSpeak or another compatible cloud platform.

The system classifies the environmental condition using predefined prototype thresholds.

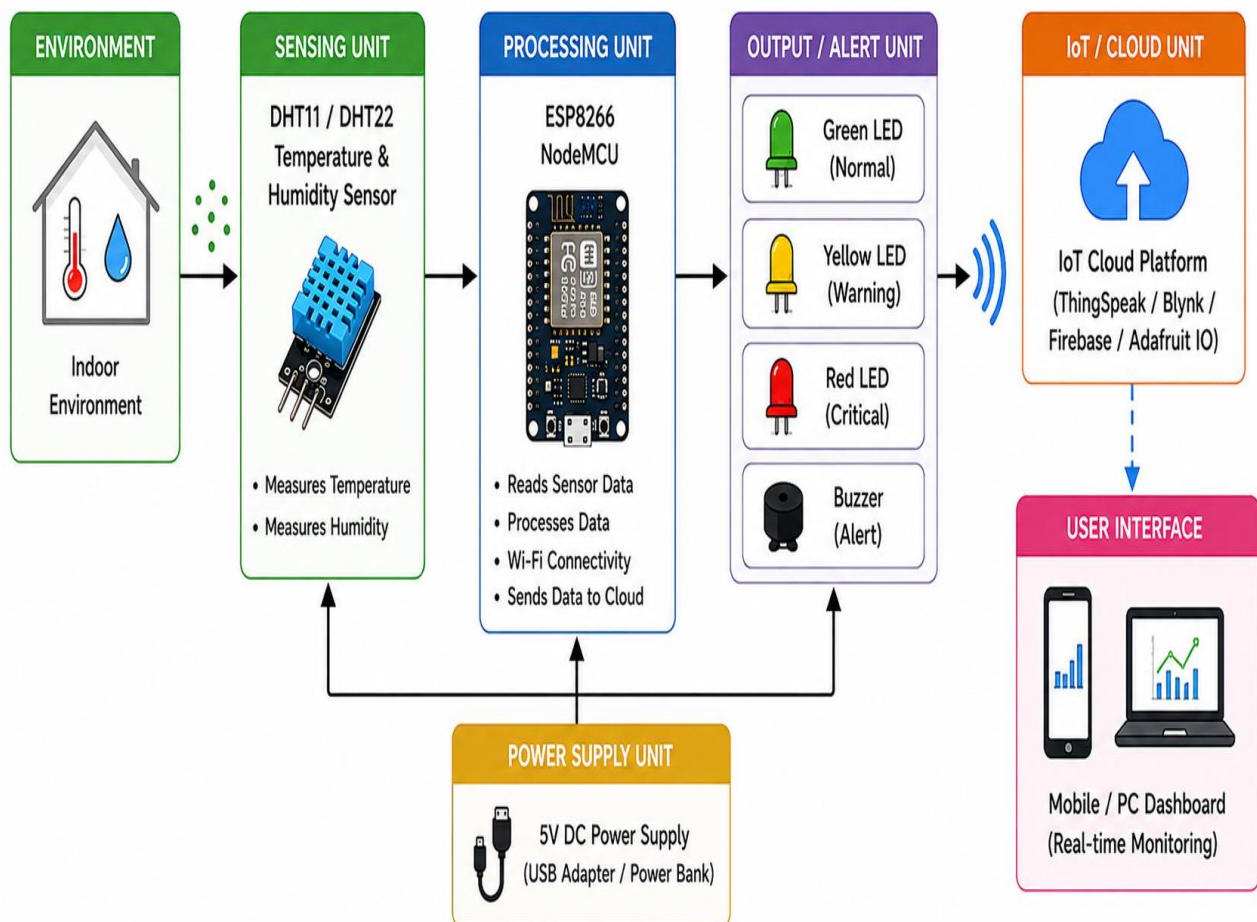
| Condition | Temperature        | Humidity | Indication       |
|-----------|--------------------|----------|------------------|
| Normal    | 20–28 °C           | 40–60%   | Green LED        |
| Warning   | 28–32 °C or 60–70% | —        | Yellow LED       |
| Critical  | >32 °C or >70%     | —        | Red LED + Buzzer |

These are demonstration thresholds for the prototype. Actual HVAC comfort limits depend on building type, occupancy, climate, equipment requirements, and applicable standards.

# System Architecture

## SYSTEM ARCHITECTURE

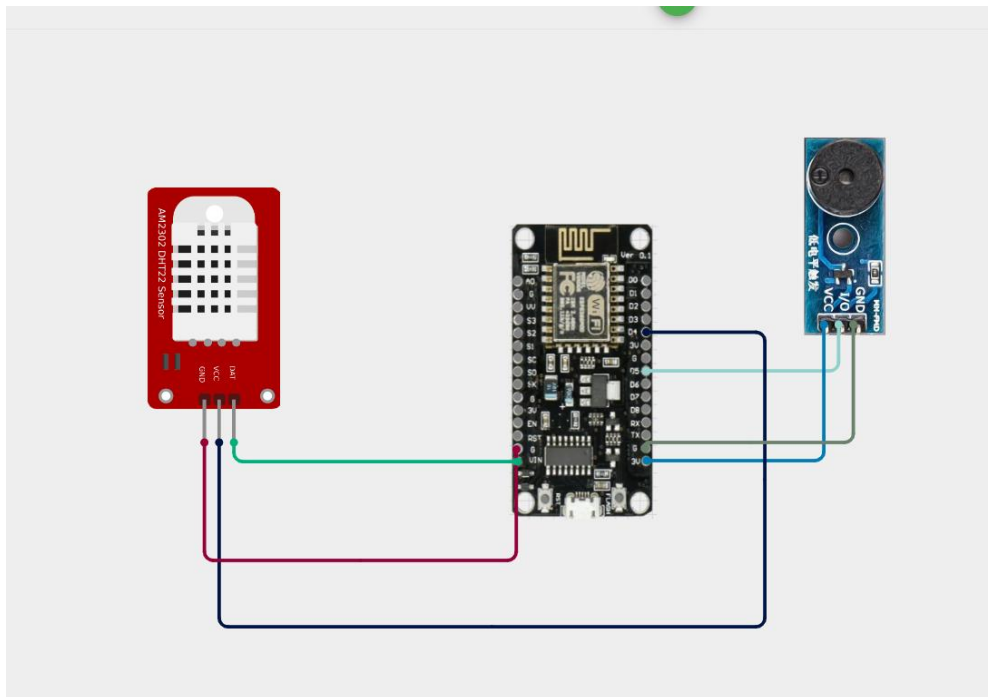
### IoT-Based HVAC Temperature and Humidity Monitoring System



## Circuit Table

| S.No. | Component   | Pin/Terminal | ESP8266 NodeMCU Connection       |
|-------|-------------|--------------|----------------------------------|
| 1     | DHT11/DHT22 | VCC          | 3.3V                             |
| 2     | DHT11/DHT22 | GND          | GND                              |
| 3     | DHT11/DHT22 | DATA         | D4 (GPIO2)                       |
| 4     | Green LED   | Anode        | D1 (GPIO5) through 220 $\Omega$  |
| 5     | Green LED   | Cathode      | GND                              |
| 6     | Yellow LED  | Anode        | D2 (GPIO4) through 220 $\Omega$  |
| 7     | Yellow LED  | Cathode      | GND                              |
| 8     | Red LED     | Anode        | D5 (GPIO14) through 220 $\Omega$ |
| 9     | Red LED     | Cathode      | GND                              |
| 10    | Buzzer      | Positive     | D6 (GPIO12)                      |
| 11    | Buzzer      | Negative     | GND                              |

## Circuit Design



## Algorithm

1. START
2. Initialize ESP8266 NodeMCU.
3. Initialize DHT11/DHT22 temperature and humidity sensor.
4. Initialize Green LED, Yellow LED, Red LED and Buzzer.
5. Connect ESP8266 to the Wi-Fi network.
6. Connect the system to the IoT cloud/dashboard.
7. Read temperature from the DHT sensor.
8. Read relative humidity from the DHT sensor.
9. Check whether the sensor readings are valid.
10. If temperature is between 20°C and 28°C and humidity is between 40% and 60%:
  - Turn ON Green LED.
  - Turn OFF Yellow LED.
  - Turn OFF Red LED.
  - Turn OFF Buzzer.
  - Display "NORMAL CONDITION".

11. Else if temperature is above 28°C and up to 32°C  
or humidity is above 60% and up to 70%:
  - Turn OFF Green LED.
  - Turn ON Yellow LED.
  - Turn OFF Red LED.
  - Turn OFF Buzzer.
  - Display "WARNING - HVAC ADJUSTMENT REQUIRED".
12. Else if temperature is above 32°C  
or humidity is above 70%:
  - Turn OFF Green LED.
  - Turn OFF Yellow LED.
  - Turn ON Red LED.
  - Turn ON Buzzer.
  - Display "CRITICAL CONDITION".
13. Send temperature and humidity data to the IoT cloud.
14. Display the readings on the Serial Monitor/dashboard.
15. Wait for the next measurement interval.
16. Repeat the monitoring process continuously.
17. STOP



## Coding

/\*\*\*\*\*\*

IoT-Based HVAC Temperature & Humidity Monitoring  
ESP8266 NodeMCU + DHT11 + Active-LOW Buzzer

\*\*\*\*\*/

```
#include <ESP8266WiFi.h>
#include "AdafruitIO_WiFi.h"
#include <DHT.h>
```

```
//----- WiFi -----
#define WIFI_SSID "alexa 007"
#define WIFI_PASS "7871976757"
```

```
//----- Adafruit IO -----
#define IO_USERNAME "madhan_2006"
#define IO_KEY "aio_Ydvh727M22rGsea2zvl7Apeq6Ui1"
```

```
AdafruitIO_WiFi io(IO_USERNAME, IO_KEY, WIFI_SSID,
WIFI_PASS);
```

```
// Adafruit Feeds
AdafruitIO_Feed *temperatureFeed = io.feed("temperature");
AdafruitIO_Feed *humidityFeed    = io.feed("humidity");
AdafruitIO_Feed *hvacFeed        = io.feed("hvac-status");
```

```
//----- DHT -----
#define DHTPIN 14    // D5 = GPIO14
#define DHTTYPE DHT11
```

```
DHT dht(DHTPIN, DHTTYPE);
```

```
//----- Buzzer -----
#define BUZZER_PIN 12 // D6 = GPIO12
```

```
void setup() {
```

```
Serial.begin(115200);  
delay(1000);
```

```
dht.begin();
```

```
pinMode(BUZZER_PIN, OUTPUT);  
digitalWrite(BUZZER_PIN, HIGH); // Active LOW buzzer -> OFF
```

```
Serial.println("Connecting to Adafruit IO...");
```

```
io.connect();
```

```
while(io.status() < AIO_CONNECTED){  
  Serial.print(".");  
  delay(500);  
}
```

```
Serial.println();  
Serial.println("Adafruit IO Connected!");  
}
```

```
void loop() {
```

```
  io.run();
```

```
  float temp = dht.readTemperature();  
  float hum  = dht.readHumidity();
```

```
  if(isnan(temp) || isnan(hum)){  
    Serial.println("DHT Error");  
    delay(2000);  
    return;  
  }
```

```
  String hvacStatus;
```

```

// HVAC Decision Logic
if(temp > 30 || hum > 70){
    hvacStatus = "COOL";
    digitalWrite(BUZZER_PIN, LOW);    // ON (Active LOW)
}
else if(temp < 20){
    hvacStatus = "HEAT";
    digitalWrite(BUZZER_PIN, LOW);    // ON
}
else{
    hvacStatus = "OPTIMAL";
    digitalWrite(BUZZER_PIN, HIGH);    // OFF
}

Serial.println("-----");
Serial.print("Temperature : ");
Serial.print(temp);
Serial.println(" °C");

Serial.print("Humidity   : ");
Serial.print(hum);
Serial.println(" %");

Serial.print("HVAC Status : ");
Serial.println(hvacStatus);

// Upload to Adafruit IO
temperatureFeed->save(temp);
humidityFeed->save(hum);
hvacFeed->save(hvacStatus);

delay(5000);
}

```

## **System Working**

The system begins by initializing the ESP8266, DHT sensor, LEDs, and buzzer.

The ESP8266 connects to the available Wi-Fi network. The DHT sensor then continuously measures the surrounding temperature and relative humidity.

The ESP8266 receives the sensor readings and compares them with the predefined thresholds.

### **Normal Condition**

If the temperature and humidity remain within the normal range:

Green LED → ON

The HVAC environment is considered to be within the prototype's desired monitoring range.

### **Warning Condition**

If temperature or humidity begins to exceed the desired range:

Yellow LED → ON

This indicates that HVAC adjustment may be required.

### **Critical Condition**

If the temperature or humidity exceeds the critical threshold:

Red LED → ON

Buzzer → ON

The system generates an alert indicating an abnormal environmental condition.

At the same time, the ESP8266 can transmit the readings through Wi-Fi to an IoT platform, where users can monitor the HVAC environment remotely.

## Bill of Materials

| S.No. | Component          | Specification          | Quantity | Purpose                            |
|-------|--------------------|------------------------|----------|------------------------------------|
| 1     | ESP8266 NodeMCU    | Wi-Fi Microcontroller  | 1        | Main Controller & IoT Connectivity |
| 2     | DHT11/DHT22 Sensor | Temperature & Humidity | 1        | Environmental Monitoring           |
| 3     | Green LED          | 5 mm                   | 1        | Normal Indication                  |
| 4     | Yellow LED         | 5 mm                   | 1        | Warning Indication                 |
| 5     | Red LED            | 5 mm                   | 1        | Critical Indication                |
| 6     | Resistor           | 220 $\Omega$           | 3        | LED Current Limiting               |
| 7     | Buzzer             | 3.3/5 V                | 1        | Audible Alert                      |
| 8     | Breadboard         | Standard               | 1        | Circuit Assembly                   |
| 9     | Jumper Wires       | Male-Male              | 1 Set    | Circuit Connections                |
| 10    | USB Cable          | Micro USB              | 1        | Power & Programming                |

## Prototype Implementation

The prototype can be constructed using an ESP8266 NodeMCU, DHT sensor, LEDs, buzzer, breadboard, and jumper wires.

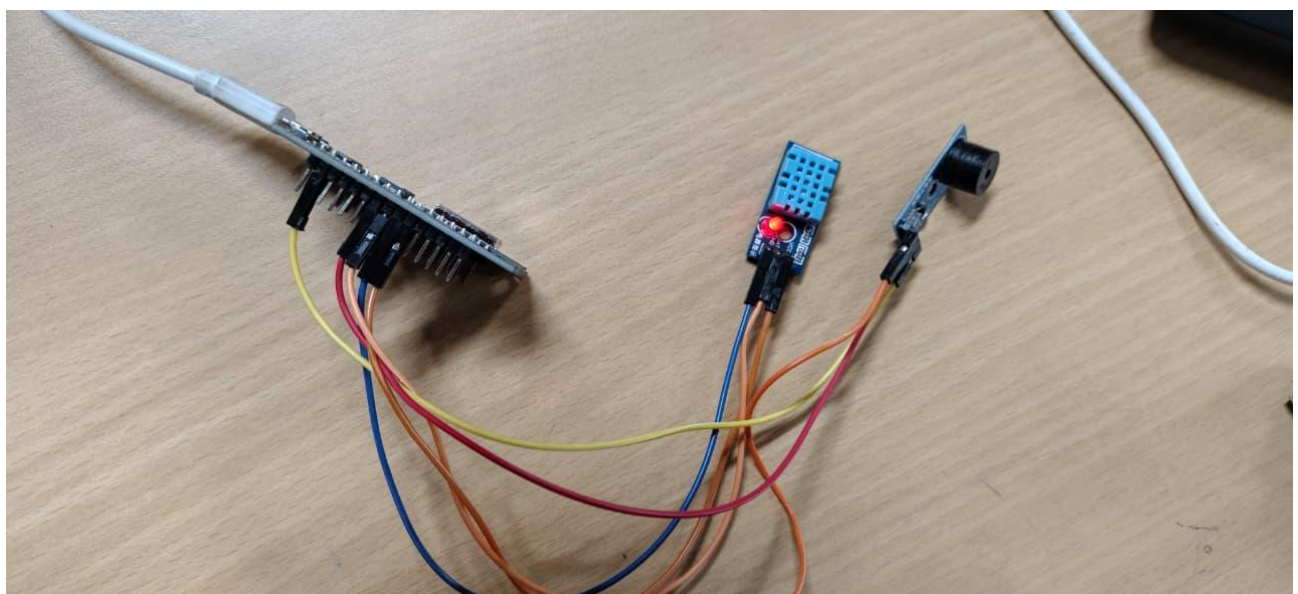
The DHT sensor is positioned inside the room or test environment where temperature and humidity need to be monitored.

The ESP8266 is connected to the sensor and output indicators. The program is uploaded through the Arduino IDE.

After powering the system, the ESP8266 connects to Wi-Fi. The sensor begins measuring environmental conditions at regular intervals.

The temperature and humidity values are displayed through the Serial Monitor and can also be transmitted to an IoT dashboard.

As the environmental conditions change, the LEDs provide local status information and the buzzer provides a critical warning.



## Results & Performance Analysis

The proposed system continuously monitors temperature and humidity and successfully identifies different environmental conditions.

| Parameter              | Result           |
|------------------------|------------------|
| Controller             | ESP8266 NodeMCU  |
| Sensor                 | DHT11/DHT22      |
| Temperature Monitoring | Yes              |
| Humidity Monitoring    | Yes              |
| Wi-Fi Connectivity     | Yes              |
| IoT Monitoring         | Yes              |
| Normal Alert           | Green LED        |
| Warning Alert          | Yellow LED       |
| Critical Alert         | Red LED + Buzzer |
| Remote Monitoring      | Supported        |
| Continuous Monitoring  | Yes              |

## Performance Analysis

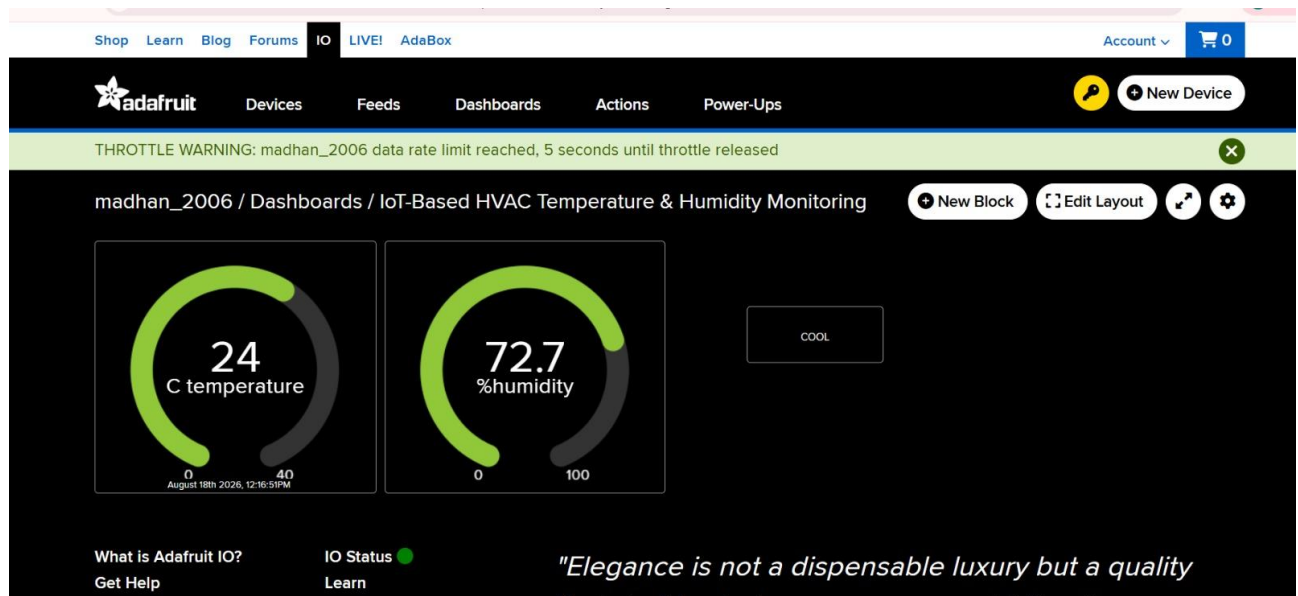
The system provides real-time environmental monitoring and allows users to observe temperature and humidity remotely.

The use of ESP8266 eliminates the need for a separate Wi-Fi module, reducing the number of components required.

The system can help identify periods where environmental conditions move outside the desired range.

This information can support HVAC optimization by enabling better scheduling, setpoint adjustment, and fault detection.

However, actual energy savings depend on how the monitoring system is integrated with the HVAC control system. Monitoring alone does not automatically reduce energy consumption.



```
-----
Temperature : 27.40 °C
Humidity    : 72.90 %
HVAC Status : COOL
-----
Temperature : 27.40 °C
Humidity    : 72.70 %
HVAC Status : COOL
-----
Temperature : 27.40 °C
Humidity    : 72.00 %
HVAC Status : COOL
-----
Temperature : 27.40 °C
Humidity    : 71.80 %
HVAC Status : COOL
-----
Temperature : 27.30 °C
Humidity    : 69.80 %
HVAC Status : OPTIMAL
-----
Temperature : 27.20 °C
Humidity    : 68.70 %
HVAC Status : OPTIMAL
-----
Temperature : 27.10 °C
Humidity    : 69.20 %
HVAC Status : OPTIMAL
-----
Temperature : 27.00 °C
Humidity    : 67.60 %
HVAC Status : OPTIMAL
-----
Temperature : 26.70 °C
Humidity    : 66.50 %
HVAC Status : OPTIMAL
```